

AI 代码信任度实验

1. 实验说明

本实验选择 `AdminQuestServiceImpl.reviewQuest` 作为对象，功能涉及管理员权限、任务状态流转、审核记录落库、任务下架后的接取记录处理。实验旨在检验 AI 是否能生成可直接用的代码。

2. 步骤一：AI 直接生成

2.1 Prompt

请基于 Spring Boot、JPA 和 Git Guild 当前 P3 设计，实现管理员审核任务功能。

业务背景：

- 任务 `Quest` 有状态：DRAFT、PENDING_ADMIN_REVIEW、PUBLISHED、IN_PROGRESS、REJECTED、CLOSED。
- 管理员可以对任务执行三种决策：APPROVE_PUBLISH、REJECT_PUBLISH、TAKE_DOWN。
- APPROVE_PUBLISH 和 REJECT_PUBLISH 只能处理 PENDING_ADMIN_REVIEW 状态的任务。
- TAKE_DOWN 可以处理任意非 CLOSED 状态任务。
- 每次审核需要生成 `AdminReviewRecord`。
- 只有 ADMIN 角色可以审核。

输出 `AdminQuestServiceImpl.reviewQuest` 的核心代码，包含异常处理和状态更新。

2.2 AI 原始输出

```
@Transactional
public AdminReviewResponse reviewQuest(Long questId, Long adminId,
AdminReviewRequest request) {
    User admin = userRepository.findById(adminId).orElseThrow();
    Quest quest = questRepository.findById(questId).orElseThrow();

    if (request.getDecision() == AdminDecision.APPROVE_PUBLISH) {
        quest.setStatus(QuestStatus.PUBLISHED);
    } else if (request.getDecision() == AdminDecision.REJECT_PUBLISH) {
        quest.setStatus(QuestStatus.REJECTED);
    } else if (request.getDecision() == AdminDecision.TAKE_DOWN) {
        quest.setStatus(QuestStatus.CLOSED);
    }

    AdminReviewRecord record = new AdminReviewRecord(
```

```
        quest,
        admin,
        request.getDecision(),
        request.getReason(),
        request.isVisibleToPublisher());
reviewRecordRepository.save(record);
questRepository.save(quest);

return new AdminReviewResponse(
    record.getAdminReviewId(),
    quest.getQuestId(),
    admin.getUserId(),
    record.getDecision(),
    record.getReason(),
    quest.getStatus(),
    record.getReviewedAt());
}
```

2.3 AI 生成代码运行测试

实验时将 AI 直出方法体临时替换到：

```
backend/src/main/java/com/gitguild/backend/quest/service/AdminQuestServiceImpl
.java
```

然后执行：

```
cd backend
.\mvnw.cmd clean verify
```

测试结果：

检查项	结果	说明
是否能编译	是	Maven 编译通过
项目是否能运行到测试阶段	是	Spring Boot 测试上下文启动成功
是否通过当前测试	否	Tests run: 106, Failures: 4, Errors: 0, Skipped: 7
失败集中点	见说明	非管理员审核、非法状态审批、重复下架、下架未取消 ACTIVE 接取记录

结论：AI 直出版本能够编译并运行到测试阶段，但在当前 P4 单元测试和集成测试下无法全部通过。

3. 步骤二：人工检查

3.1 是否符合 P3 详细设计

不完全符合。P3 API 规范和数据库设计要求管理员审核需要有明确角色权限、业务错误码、审核记录和任务状态约束。AI 代码只修改状态并保存记录，没有校验 `admin.getRole()`，也没有使用项目统一的 `BusinessException` 错误响应。

3.2 是否处理常见异常或边界情况

处理不足。AI 代码使用 `orElseThrow()` 默认异常，找不到用户或任务时会产生不可控异常响应；同时没有限制 `APPROVE_PUBLISH`、`REJECT_PUBLISH` 只能作用于 `PENDING_ADMIN_REVIEW` 状态，也没有阻止对 `CLOSED` 任务重复下架。

3.3 是否存在明显 Bug 或不合理逻辑

存在。AI 代码直接调用 `quest.setStatus(...)`，绕过了 `Quest.approve()`、`Quest.reject()`、`Quest.takeDown()` 等领域方法。这样会导致发布时间等领域副作用缺失，也会让状态规则分散在 `Service` 中。更严重的是，执行 `TAKE_DOWN` 时没有取消仍处于 `ACTIVE` 状态的接取记录，可能出现任务已关闭但仍有人处于接取中的不一致状态。

3.4 人工检查发现的主要问题

主要问题	影响	修复方式
缺少管理员角色校验和统一错误码	非管理员可能绕过前端调用审核接口；异常响应不符合 API 规范	查询用户后校验 <code>UserRole.ADMIN</code> ，失败时抛出 <code>FORBIDDEN</code> ；用户或任务不存在时抛出 <code>USER_NOT_FOUND</code> 、 <code>QUEST_NOT_FOUND</code>
缺少状态机约束	已发布、进行中、已关闭任务可能被错误通过或驳回	使用 <code>Quest.canBeApprovedOrRejected()</code> 、 <code>Quest.canBeTakenDown()</code> 守卫状态流转
下架未联动接取记录	<code>CLOSED</code> 任务仍可能存在 <code>ACTIVE</code> 接取记录，影响后续提交、审核和演示数据一致性	<code>TAKE_DOWN</code> 时查询 <code>ACTIVE</code> 接取记录，调用 <code>QuestAssignment.cancel()</code> 并保存

3.5 人工修复后重新运行

人工修复后的核心实现保留在 `AdminQuestServiceImpl.reviewQuest` 和 `applyDecision` 中。主要变化如下：

```
@Transactional
public AdminReviewResponse reviewQuest(Long questId, Long adminId,
AdminReviewRequest request) {
    User admin = userRepository.findById(adminId)
        .orElseThrow(() -> new BusinessException("USER_NOT_FOUND",
HttpStatus.NOT_FOUND, "用户不存在", "userId=" + adminId));
    if (admin.getRole() != UserRole.ADMIN) {
        throw new BusinessException("FORBIDDEN", HttpStatus.FORBIDDEN, "当前用
户无权限", "只有管理员可以审核任务");
    }

    Quest quest = questRepository.findById(questId)
        .orElseThrow(() -> new BusinessException("QUEST_NOT_FOUND",
HttpStatus.NOT_FOUND, "任务不存在", "questId=" + questId));

    applyDecision(quest, request.getDecision());

    AdminReviewRecord record = new AdminReviewRecord(
        quest, admin, request.getDecision(), request.getReason(),
request.isVisibleToPublisher());
    reviewRecordRepository.save(record);
    questRepository.save(quest);

    return new AdminReviewResponse(
        record.getAdminReviewId(),
        quest.getQuestId(),
        admin.getUserId(),
        record.getDecision(),
        record.getReason(),
        quest.getStatus(),
        record.getReviewedAt());
}
```

修复后重新执行：

```
cd backend
.\mvnw.cmd clean verify
```

运行结果：

检查项	结果
是否能编译	是
项目是否能运行到测试阶段	是
是否通过当前测试	是
测试统计	Tests run: 106, Failures: 0, Errors: 0, Skipped: 7
构建结果	BUILD SUCCESS

4. 步骤三：实验记录表

指标	AI 直出	人工审查修复后
编译是否通过	通过。mvnw clean verify 进入编译和测试阶段	通过。恢复人工修复版本后重新编译成功
功能是否可运行	正常骨架可运行，但人工审查发现越权、非法状态、下架联动等业务路径不可信	可以，正常审批、驳回、下架、权限错误、状态错误均有明确处理逻辑
测试是否通过	未通过当前测试：Tests run: 106, Failures: 4, Errors: 0, Skipped: 7	通过当前测试：Tests run: 106, Failures: 0, Errors: 0, Skipped: 7
主要问题/修复说明	缺少 ADMIN 角色校验；缺少状态机约束；下架不取消 ACTIVE 接管记录；默认异常不符合 API 规范	增加 BusinessException 和错误码；通过 Quest 领域方法约束状态流转；下架时取消 ACTIVE 接管记录

5. 实验结论

AI 可以快速生成“看起来完整”的服务层骨架，但 AI 直出版本被权限、状态机和下架联动相关测试拦截。因此，“能编译、能进入测试阶段”不能直接等价于“业务可信”。

本次实验中，AI 的主要问题不是语法错误，而是没有真正落实 P3 设计中的权限、状态机、异常码和跨表一致性要求。对 Git Guild 这类包含任务状态流转和角色边界的系统，AI 代码必须经过人工按设计文档逐项审查，尤其要检查：

- 是否落实角色权限。
- 是否符合状态机和异常流。
- 是否使用统一错误响应。
- 是否维护跨表数据一致性。

- 是否绕过领域对象中已有的业务方法。

AI 适合生成初稿和样板代码，但不能作为后端核心业务逻辑的直接合入依据。